MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Compte Rendu – TP5 : Apprentissage auto-supervisé

Apprentissage profond pour la vision (IFLCE015) – Maksym DOLHOV – 2026

Code : https://github.com/3v3r51nc3/tp5-self-supervised-learning

1 Objectif et travail réalisé

On compare des **tâches prétexte** auto-supervisées : une représentation est apprise sans étiquettes, puis évaluée par un classifieur linéaire sur features figées (*linear probe*). Ont été faits : les démos fournies (Context Encoder, Rotation, SimCLR) sur CIFAR10 ; leur comparaison dans un tableau et un graphique ; la même comparaison sur **CIFAR100** ; et l'implémentation d'une nouvelle tâche, la **position relative d'un patch** (Doersch et al., 2015), intégrée à la bibliothèque `deepul_helper` (`ckurtz/cs294-158-ssl`) et évaluée sur les deux jeux de données. Les expériences ont tourné sur Google Colab (Tesla T4, PyTorch). Le serveur officiel de CIFAR étant très lent, CIFAR100 a été reconstruit depuis la copie Hugging Face au format torchvision (pixels et étiquettes vérifiés identiques).

Fichiers : `src/patch_position.py` (tâche), `src/apply_patches.py` (intégration + CIFAR100), `notebooks/tp5_cifar100_run.ipynb` (entraînements et tableau final).

2 Tâche prétexte : position relative d'un patch

L'image est une grille 3×3 ; le patch central est le *query*, un des 8 voisins est tiré au hasard, et le réseau prédit sa position (classification à 8 classes, hasard = 12,5 %). Choix d'implémentation :

- images 32×32 suréchantillonnées à 64×64 sur GPU (le CPU de Colab n'a que 2 cœurs), patchs de 16×16 ;
- espace de 6 px entre patchs et *jitter* de ± 2 px indépendant par patch, pour éviter le raccourci par les bords ou la continuité des lignes ;
- suppression de couleur (un canal gardé, deux remplacés par du bruit gaussien) contre l'aberration chromatique ;
- encodeur siamois : même tronc (6 blocs conv-BN-ReLU, 64-64-128-128-256-256), concaténation des deux features, tête MLP $2048 \to 512 \to 8$, entropie croisée ;
- pour le linear probe, l'image entière passe dans le même tronc (pooling adaptatif 2×2, dimension 1024).

La perte initiale vaut $\approx \ln 8 = 2{,}07$. La précision prétexte atteint $\approx 43\,\%$ sur CIFAR10 (100 époques) et passe de 20 % à 27 % en 3 époques sur CIFAR100 : la tâche est apprise sans être triviale.

3 Protocole et résultats

Métrique : précision top-1/top-5 du classifieur linéaire sur le jeu de test. **CIFAR10** : Context Encoder, Rotation et SimCLR sont les modèles *pré-entraînés fournis* avec le TP (nombre

d'époques inconnu, long) ; la position de patch a été entraînée par mes soins (100 époques). **CIFAR100** : même code, mais **3 époques par tâche**, même budget pour les quatre (temps GPU limité).

Tâche	C10 top-1	C10 top-5	C100 top-1	C100 top-5
Context Encoder	45,77	90,29	13,09	33,89
Rotation	79,91	99,12	28,69	57,27
SimCLR	92,84	99,86	12,29	34,43
Position de patch	59,92	94,97	8,56	25,35

TABLE 1 – Précision (%) du linear probe sur le test (C100 : 3 époques seulement).

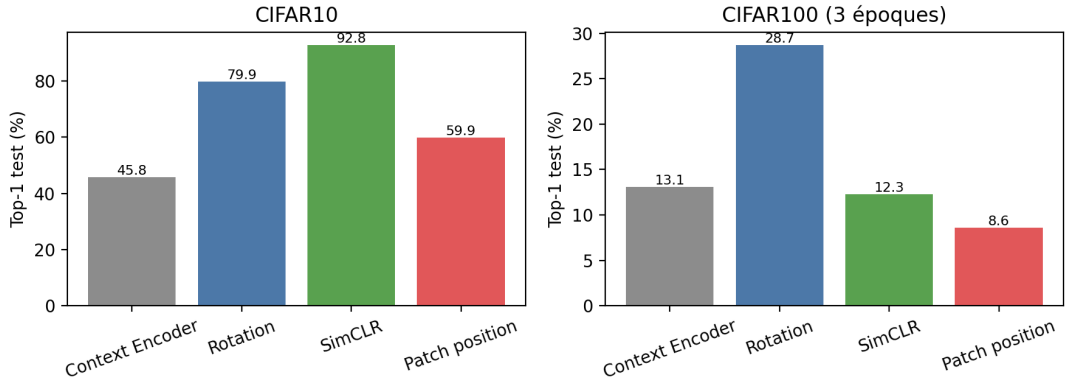


FIGURE 1 – Comparaison des quatre tâches prétexte (top-1 test).

4 Analyse

CIFAR10. SimCLR domine (92,84), puis Rotation (79,91), position de patch (59,92) et Context Encoder (45,77). L'apprentissage contrastif impose l'invariance aux augmentations qui préservent la classe ; la rotation demande de reconnaître l'objet et son orientation, proche de la classification ; la position de patch exige de comprendre un peu la structure des objets mais peut être résolue en partie par des indices de bas niveau ; la reconstruction du Context Encoder favorise textures et couleurs, peu liées à la classe.

CIFAR100. Avec 3 époques, tous les scores sont bas et le classement change : Rotation (28,69) est loin devant, SimCLR (12,29) rejoint le Context Encoder et la position de patch est dernière (8,56). Les pertes de SimCLR et de la position de patch baissaient encore vite : ces méthodes demandent bien plus d'époques, contrairement à la rotation. Sur CIFAR10, la position de patch passe de 51,05 % (20 époques) à 59,92 % (100), ce qui va dans ce sens (non testé sur CIFAR100).

À budget égal de 3 époques, la comparaison reste équitable sur le plan des données, du batch et du linear probe : seule la rotation se détache clairement, tandis que Context Encoder et SimCLR sont trop proches pour être départagés (un seul entraînement par tâche). Attention : à nombre d'époques égal, le coût de calcul par époque diffère fortement d'une tâche à l'autre (SimCLR est bien plus coûteux que la position de patch) et les hyperparamètres n'ont pas été réglés ; ce classement décrit donc la vitesse d'apprentissage initiale, pas la qualité finale des représentations.

5 Limites

Ressources de calcul limitées : je ne disposais que d'un GPU Colab (Tesla T4, 2 cœurs CPU, sessions de durée limitée), ce qui a empêché d'entraîner longtemps. Entraîner les quatre modèles est coûteux pour ce matériel, d'où les 3 époques par tâche sur CIFAR100 et l'absence d'étude d'ablation ou de réglage d'hyperparamètres. Conséquence : budgets inégaux (3 époques sur CIFAR100 contre des modèles pré-entraînés longuement sur CIFAR10), les colonnes ne sont pas comparables entre elles. Hyperparamètres non optimisés (SimCLR : batch 128), un seul entraînement par configuration (pas de barres d'erreur), et classifieur linéaire entraîné en même temps que l'encodeur (conception de la bibliothèque).

6 Utilisation d'outils d'IA

Un assistant IA (Claude) a été utilisé en appui pour : (1) le débogage et l'intégration dans `deepul_helper`, (2) l'écriture des scripts de téléchargement et de préparation des données (le serveur officiel de CIFAR étant trop lent, CIFAR10 et CIFAR100 ont dû être récupérés autrement), (3) la mise en forme LaTeX du compte rendu. La conception de l'expérience, l'exécution des entraînements sur Colab, l'interprétation des résultats et la rédaction de l'analyse sont de mon fait.

7 Conclusion

La position relative de patch est implémentée dans la bibliothèque et comparée aux trois autres tâches sur deux jeux de données. Elle apprend des features utiles (59,92 % sur CIFAR10, loin des 10 % du hasard) mais reste sous la Rotation et SimCLR et au-dessus du Context Encoder. Piste principale : entraîner les quatre tâches sur CIFAR100 avec autant d'époques que sur CIFAR10.